Modelling Interactions
State

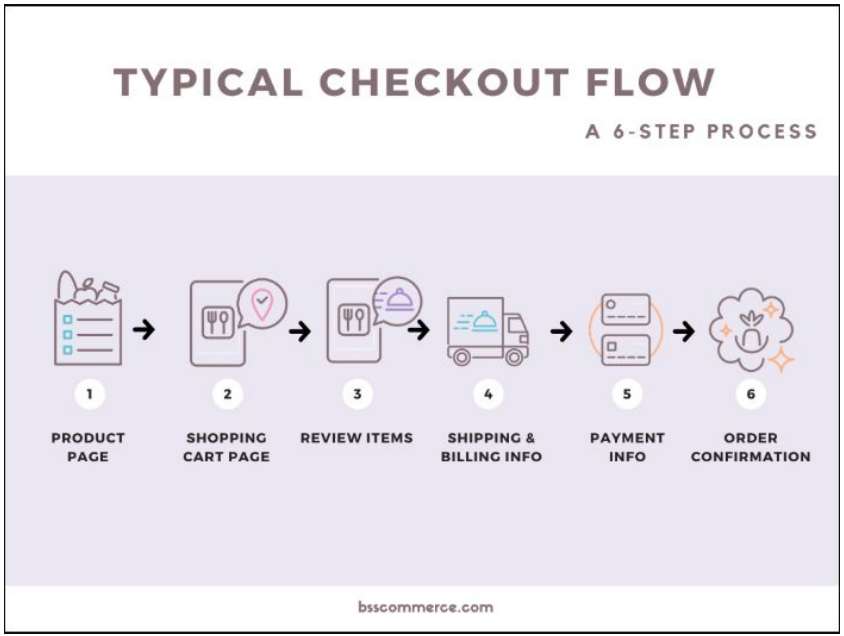
Learning Objectives

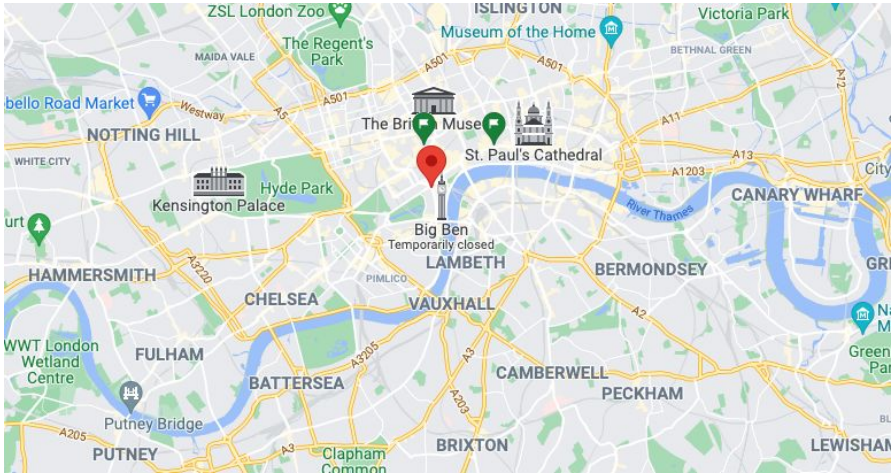
1. Explain how to model a simple user interaction
2. Use models to drive building a frontend application
3. Define state as the data the application needs to store to successfully complete a user interaction
4. Build a simple frontend application using the pattern of maintaining an encapsulated state object as the source of truth for application state

Development Process

Requirements -> stories -> domain model -> tests -> implement.

Models simplify reality to show information



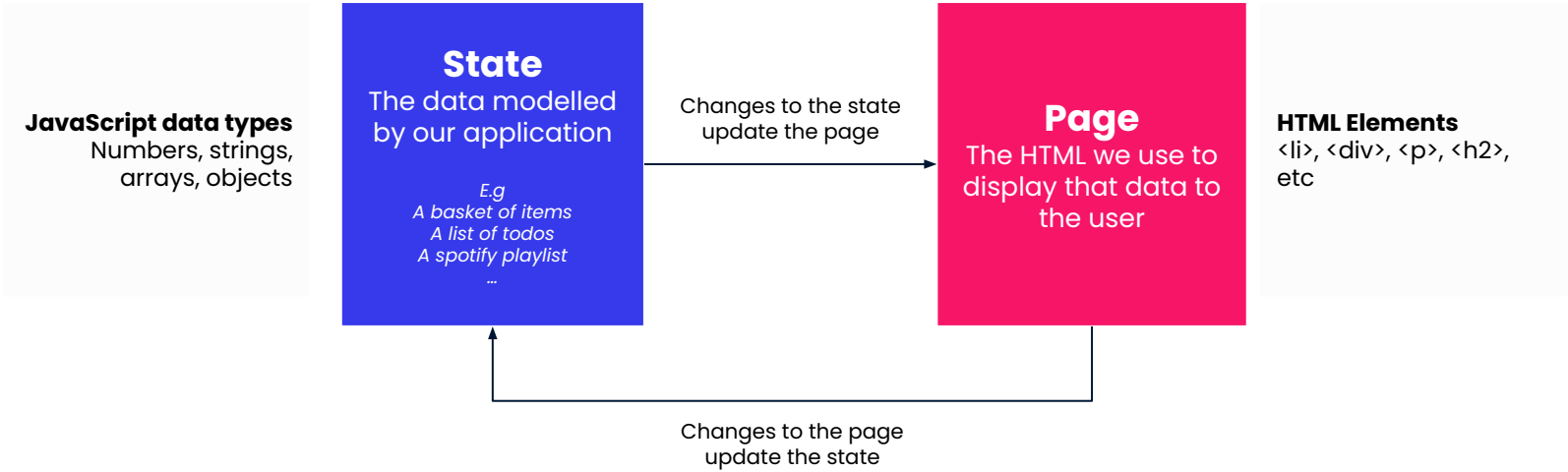


Modelling

Objects	Properties	Messages	Notes	Scenario	Output	Example
TodoList	id @Int, items @Array	create(@String)	id increments, status starts off incomplete, adds item to array		todo item	`create('hello') => {id: 1, text: "hello", status: "incomplete"}`
		showAll()			all items	`showAll() => [{id: 1, text: "hello", status: "incomplete"}]`
		setComplete(@Int)	finds item, then updates status property	item exists	updated todo item	`setComplete(1) => {id: 1, text: "hello", status: "complete"}`
				item does not exist	thrown error	`setComplete(1) => thrown error "Item not Found"`
		getByStatus(@String)			array, filtered by property status	`getByStatus("incomplete") => [{id: 1, text: "hello", status: "incomplete"}]`
		findBy(@Int)		item exists	item	`findBy(1) => {id: 1, text: "hello", status: "incomplete"}`
				item does not exist	thrown error	`findBy(1) => thrown error "Item not Found"`
		deleteBy(@Int)	finds item, then removes it from array	item exists	item	`deleteBy(@Int) => {id: 1, text: "hello", status: "incomplete"}`
				item does not exist	thrown error	`deleteBy(@Int) => thrown error "Item not Found"`

Why do we need a model?

Let's consider an example:
https://github.com/boolean-uk/js-dom-state-favourite-foods


JavaScript data types
Numbers, strings, arrays, objects
State
The data modelled by our application
E.g
A basket of items
A list of todos
A spotify playlist
...
Changes to the state update the page
Page
The HTML we use to display that data to the user
HTML Elements
, , <p>, <h2>, etc
Changes to the page update the state

Example Requirements

Standard Requirements

- User should see an input field they can type text into and a button that says submit.
- Whenever user clicks the submit button, user sees the text they typed in the input field in a list format below.

Extended Requirements

- Items in the list can be marked as high priority - these should always appear at the top of the list and be highlighted in red.
- A checkbox should be presented that allows the user to show only high priority items
- Items should be able to be marked as “archived” - when they are archived, they appear in a smaller list underneath the main list

Domain Model

Teacher demo

How do we model state in our application?

Time for some practice

TASK: Model these requirements.

> Get together in your peer groups and share screenshots in your cohort channel
- What the page looks like before and after the interaction
- What data you need to capture and store, and how.

NB: Don’t write any code!

Time for some live coding

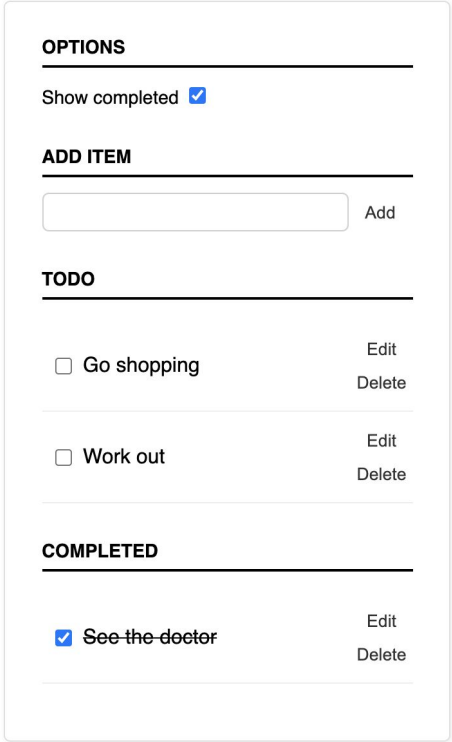
Here's a todo app, it's not fully built yet, let's explore it

Q: What's missing from it? How would you complete it?

The data our app needs

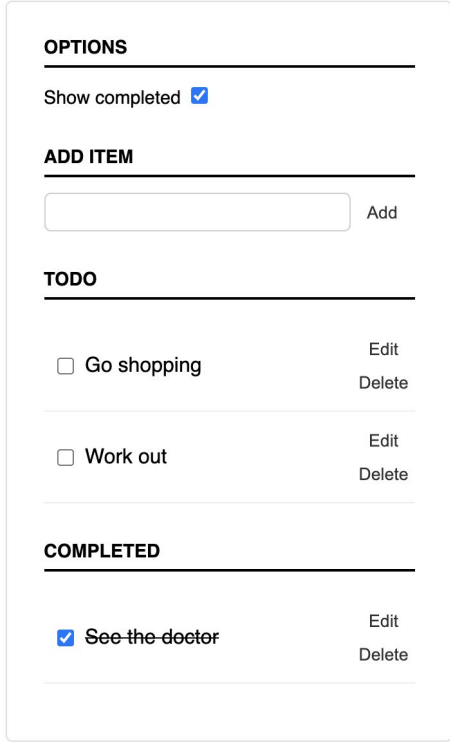
Thinking in pure **JavaScript data types**

Q: What kind of data do you need to keep track of for this app to fully work?



The data our app needs

```
let todoInput = ''
let showCompleted = true
let todos = [
    {
        text: 'Go shopping',
        completed: false
    },
    // ...etc
]
```

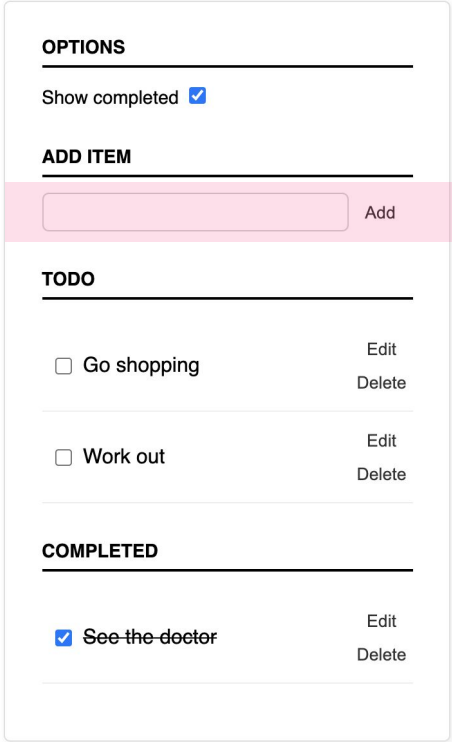


Here's one way to do it

We store a **variable** for each piece of **data** we need

The data our app needs

```
let todoInput = ''
let showCompleted = true
let todos = [
    {
        text: 'Go shopping',
        completed: false
    },
    // ...etc
]
```

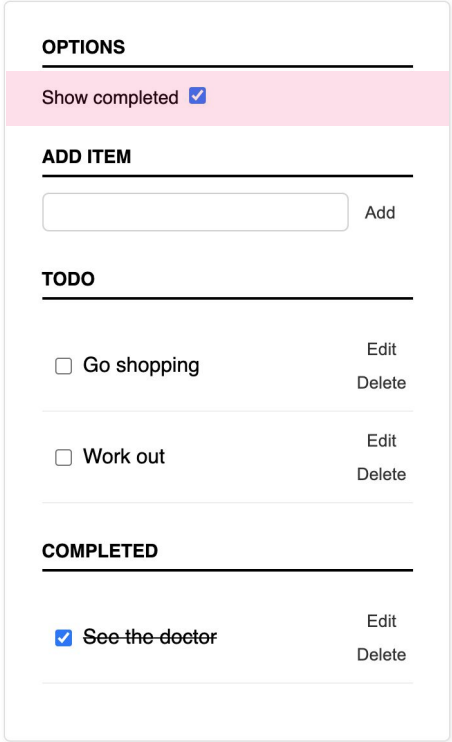


This one we may choose to store it or not

It represents the current **contents of the input**

The data our app needs

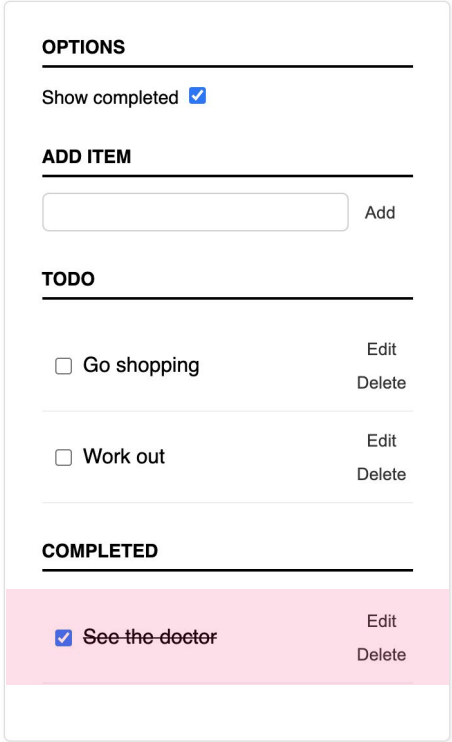
```
let todoInput = ''
let showCompleted = true
let todos = [
    {
        text: 'Go shopping',
        completed: false
    },
    // ...etc
]
```



This one is simple, we need to track whether this checkbox is **ticked or not**, so we know whether we should **show the completed tasks or not**

The data our app needs

```
let todoInput = ''
let showCompleted = true
let todos = [
    {
        text: 'Go shopping',
        completed: false
    },
    // ...etc
]
```



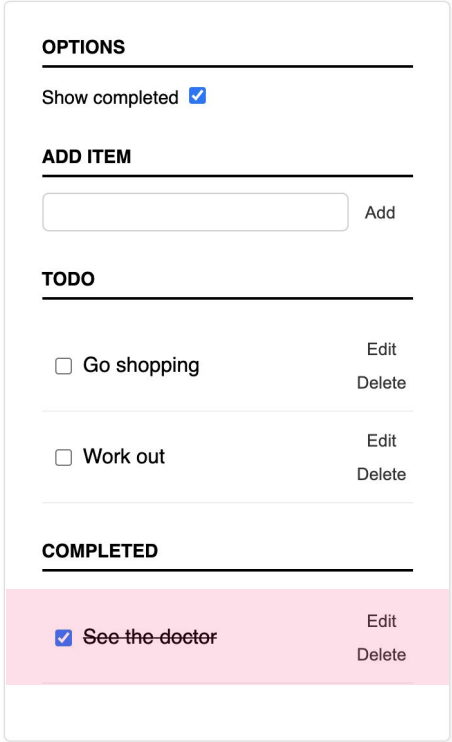
Finally, we need data about our **todos**

We know there are **multiple** todos, so we need an **array**

Q: How do we know what data each todo needs?

The data our app needs

```
let todoInput = ''
let showCompleted = true
let todos = [
    {
        text: 'Go shopping',
        completed: false
    },
    // ...etc
]
```



Each todo might be **holding more data**, but we **can't know that**, and we **don't care**, since we don't need it for our app to **work** for now

State

```
let todoInput = ''
// lots of lines of code
let showCompleted = true
// lots more lines of code
let todos = [
    {
        text: 'Go shopping',
        completed: false
    },
    // ...etc
]
```

All of this data that the app needs to function is called **state**

*Rule of thumb: keep state **local**, keep state **minimal**.*

State

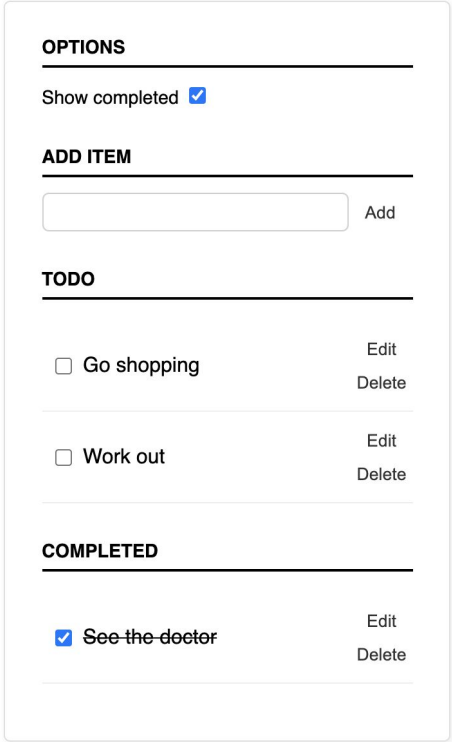
```
const state = {
    todoInput: '',
    showCompleted: true,
    todos: [
        {
            text: 'Go shopping',
            completed: false
        },
        // ...etc
    ]
}
```

Since all of this data represents our **state**, let's start by **encapsulating** it in a plain old javascript **object** with **properties** and **values**

This will make it easier to manage, and we won't risk ending up with pieces of state all over the place

Syncing state

```
const state = {
    todoInput: '',
    showCompleted: true,
    todos: [
        {
            text: 'Go shopping',
            completed: false
        },
        // ...etc
    ]
}
```

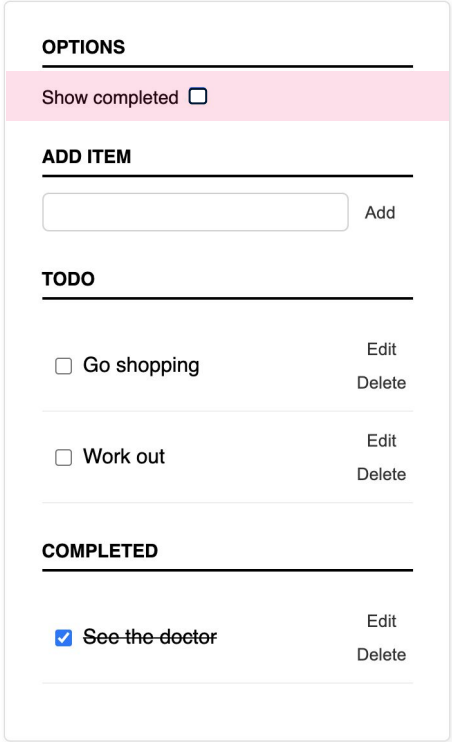


Next, we want our state to be **in sync** with the page

If **something changes on the page**, the **state should be updated**
if **something changes on the state**, the **page should be updated**

Syncing state

```
const state = {
   todoInput: '',
   showCompleted: false,
   todos: [
       {
           text: 'Go shopping',
           completed: false
       },
       // ...etc
   ]
}
```

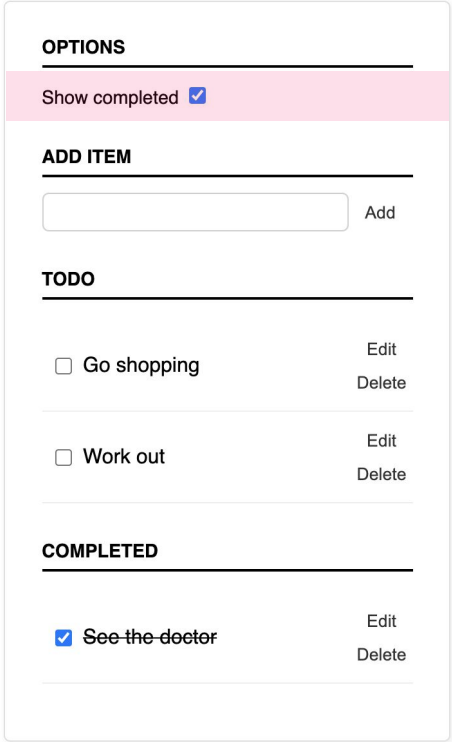


For example:

- if the **checkbox** is **unticked**
- **showCompleted** should be **false**

Syncing state

```
const state = {
   todoInput: '',
   showCompleted: true,
   todos: [
       {
           text: 'Go shopping',
           completed: false
       },
       // ...etc
   ]
}
```

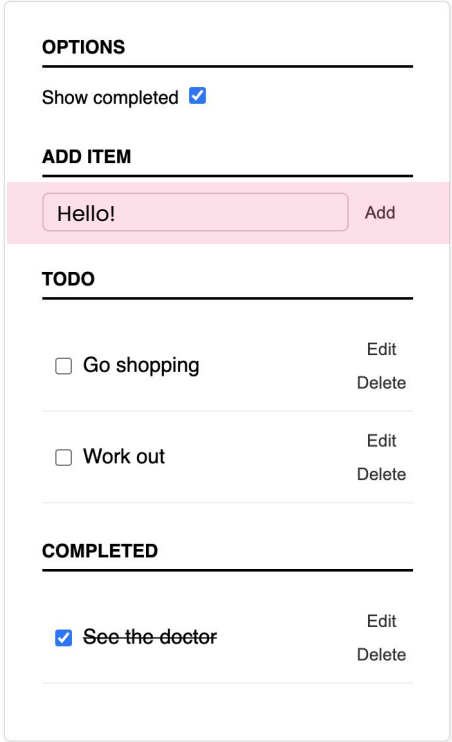


For example:

- if the **checkbox** is **ticked**
- **showCompleted** should be **true**

Syncing state

```
const state = {
   todoInput: 'Hello!',
   showCompleted: true,
   todos: [
       {
           text: 'Go shopping',
           completed: false
       },
       // ...etc
   ]
}
```

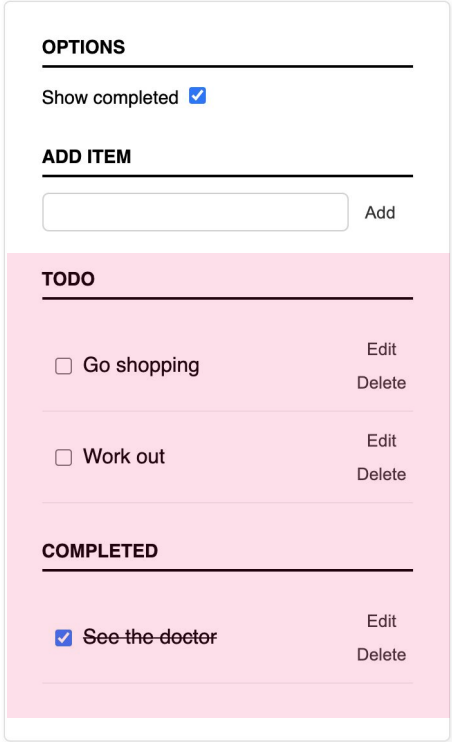


For the input, whatever **text** is **in the field**, should be **in state**

And **vice versa**

Syncing state

```
const state = {
   todoInput: '',
   showCompleted: true,
   todos: [
       {
           text: 'Go shopping',
           completed: false
       },
       // ...etc
   ]
}
```



Finally, we hold all todos **in state**

And we display them on the page **as needed**

Q: Will the page always display every todo that's in state?

The state approach

With this approach, we should:

- Treat **state,** not the elements in the DOM, as our **source of truth**
- Keep state **local**, keep state **minimal**.
- Always **change state first**
- Whenever state changes**, update the DOM** to reflect it

Time for some live coding

Let’s change our todo app to use this new approach!

Here’s a possible solution

The state approach

```
let state = { showCompleted: true }

function render() {
   renderCheckbox()
   renderInput()
   renderTodos()
}

function setStateAndRender(updatedState) {
   Object.keys(updatedState).forEach(prop =>
state[prop] = updatedState[prop])
   render()
}
```

Here's a code overview of this approach

The state approach

```
let state = { showCompleted: true }

function render() {
  renderCheckbox()
  renderInput()
  renderTodos()
}

function setStateAndRender(updatedState) {
  Object.keys(updatedState).forEach(prop =>
state[prop] = updatedState[prop])
  render()
}
```

We first store the **state** our app needs

The state approach

```
let state = { showCompleted: true }

function render() {
  renderCheckbox()
  renderInput()
  renderTodos()
}

function setStateAndRender(updatedState) {
  Object.keys(updatedState).forEach(prop =>
state[prop] = updatedState[prop])
  render()
}
```

Then we create a **render function**

This is where we describe how to put everything **on the page**, based on the **current state**

The state approach

```
let state = { showCompleted: true }

function render() {
    renderCheckbox()
    renderInput()
    renderTodos()
}

function setStateAndRender(updatedState) {
    Object.keys(updatedState).forEach(prop =>
state[prop] = updatedState[prop])
    render()
}
```

Finally, we have **setStateAndRender**

This is a helper function that will:

- **Merge** the **old and new state** in a **new object**
- **Call** the **render** function for us

The state approach

```
let state = { showCompleted: true }

function render() {
   renderCheckbox()
   renderInput()
   renderTodos()
}

function setStateAndRender(updatedState) {
   Object.keys(updatedState).forEach(prop =>
state[prop] = updatedState[prop])
   render()
}
```

With this pattern, once you've created your **state** and **render** to work together, you can just call **setStateAndRender** whenever you need to update the state, and you know the page will update as needed

The state approach

```
let state = { showCompleted: true }

function render() {
   renderCheckbox()
   renderInput()
   renderTodos()
}

function setStateAndRender(updatedState) {
   Object.keys(updatedState).forEach(prop =>
state[prop] = updatedState[prop])
   render()
}
```

This is a useful pattern that will allow us to more easily build some more complex applications, but it has a downside

Q: Can you guess what that is?

The state approach

```
let state = { showCompleted: true }

function render() {
   renderCheckbox()
   renderInput()
   renderTodos()
}

function setStateAndRender(updatedState) {
   Object.keys(updatedState).forEach(prop =>
state[prop] = updatedState[prop])
   render()
}
```

That's right, re-rendering the entire page every time is wasteful when it comes to performance!

This is no big deal for small apps, but larger apps will suffer greatly!

React uses a very similar approach to this one, but on top of it, it uses a diffing algorithm to make changes efficiently!